

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Pruebas de Software				
TÍTULO DE LA PRÁCTICA:	Depuración y Pruebas de Software				
NÚMERO DE LA PRÁCTICA:	01	AÑO LECTIVO:	2026	NRO. SEMESTRE:	VII
FECHA DE PRESENTACIÓN:	2026-04-17	HORA DE PRESENTACIÓN:	11:59 pm		
INTEGRANTE(s): - Quispe Condori, Alvaro Raul - Jara Mamani, Mariel Alisson - Arce Mayhua, Leonardo Ruben				NOTA:	Nota colocada por el docente
DOCENTE: Prof. Robert Arisaca / Prof. Diego Iquira / Prof. Lino Pinto					

SOLUCIÓN Y RESULTADOS

I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS

Ejercicio 1.1 – Depuración: algoritmo Shell Sort

El programa `shell_sort.py` implementa el algoritmo Shell Sort con la secuencia de incrementos de Knuth [1]. Se creó el proyecto `/LAB_01/` en VS Code y ejecutó el programa con los argumentos indicados en la guía, obteniendo los siguientes resultados:

Comando ejecutado	Salida obtenida	Estado
<code>python shell_sort.py 9 7 3 11</code>	Output: 3 7 9 11	Correcto
<code>python shell_sort.py 14 2 7 1</code>	Output: 2 7 14 1	Incorrecto
<code>python shell_sort.py 14 \$ 7 A</code>	Output: 0 7 14 0	Incorrecto

Identificación del defecto. El error se ubica en la función `main()` en la asignación del tamaño del arreglo:

- Código erróneo: `size = len(a) - 1`
- Código correcto: `size = len(a)`

El valor `size` delimita el bucle `for i in range(h, size)` dentro de `shell_sort`. Con `size = len(a) - 1`, el elemento en el último índice nunca es considerado en el ordenamiento. Por ejemplo, para la entrada `14 2 7 1`, el arreglo queda `[2, 7, 14, 1]` porque el elemento `1` en el índice `3` no es procesado.

Proceso de depuración en VS Code: Se configuró `launch.json` con `args: ["14", "2", "7", "1"]`, se colocaron breakpoints en las líneas `30`, `37` y `38`, y se presionó `F5`. El panel de variables mostró `size = 3` cuando el valor correcto es `4`.

```

28 def main():
29     args = sys.argv[1:]
30     a = []
31     for arg in args:
32         try:
33             a.append(int(arg))
34         except ValueError:
35             a.append(0)
36
37     size = len(a)-1
38     shell_sort(a, size)
39     print("Output:", end=" ")
40     for num in a:
41         print(num, end=" ")
42     print()

```

Locals

- > a = [14, 2, 7, 1]
- arg = '1'
- > args = ['14', '2', '7', '1']
- size = 3

Globals

Figura 1 – Panel de variables del depurador mostrando size = 3 (valor incorrecto).

Corrección aplicada. Se reemplazó `size = len(a) - 1` por `size = len(a)`. Con la corrección, `python shell_sort.py 14 2 7 1` produce correctamente Output: 1 2 7 14.

```

import sys

def shell_sort(a, size):
    h = 1
    while h <= size // 3:
        h = h * 3 + 1
    while h != 1:
        h //= 3
        for i in range(h, size):
            v = a[i]
            j = i
            while j >= h and a[j - h] > v:
                a[j] = a[j - h]
                j -= h
            if i != j:
                a[j] = v

def main():
    args = sys.argv[1:]
    a = []
    for arg in args:
        try:
            a.append(int(arg))
        except ValueError:
            a.append(0)

    size = len(a) # FIX: el codigo original tenia len(a) - 1
    shell_sort(a, size)
    print("Output: ", end=" ")
    for num in a:
        print(num, end=" ")
    print()

if __name__ == "__main__":
    main()

```

```

28 def main():
29     args = sys.argv[1:]
30     a = []
31     for arg in args:
32         try:
33             a.append(int(arg))
34         except ValueError:
35             a.append(0)
36
37     size = len(a)
38     shell_sort(a, size)
39     print("Output:", end=" ")
40     for num in a:
41         print(num, end=" ")
42     print()

```

Locals

- > a = [14, 2, 7, 1]
- > arg = '1'
- > args = ['14', '2', '7', '1']
- > size = 4

Globals

Figura 2 – Panel de variables con el valor corregido size = 4.

Ejercicio 1.2 – Pruebas: Clasificación de Triángulos [2]

Se creó el archivo triangle.py en VS Code con el código del docente.

Paso 2 – Ingreso de valores inválidos. Al ejecutar el código original (sin validación de lados positivos) con entradas como (3, 0, 5) o (3, 4, -5), el programa clasifica el triángulo incorrectamente en vez de declararlo inválido. Esto evidencia un defecto: la versión original omite verificar que los lados sean positivos.

Paso 3 – Respuesta a los 13 criterios de Myers [2].

#	¿Se tiene un caso de prueba que...?	¿Cumple?
1	Represente un triángulo escaleno válido.	Sí – (3, 4, 5)
2	Represente un triángulo equilátero válido.	Sí – (7, 7, 7)
3	Represente un triángulo isósceles válido.	Sí – (5, 5, 8)
4	Pruebe las tres permutaciones de isósceles (a=b, a=c, b=c).	Sí – (3,3,4); (3,4,3); (4,3,3)
5	Un lado con valor cero.	Sí – (3, 0, 5)
6	Un lado con valor negativo.	Sí – (3, 4, -5)
7	Tres enteros positivos donde la suma de dos lados iguale al tercero.	Sí – (1, 2, 3)
8	Las tres permutaciones del caso 7 (a+b=c, a+c=b, b+c=a).	Sí – (1,2,3); (1,3,2); (3,1,2)
9	Tres enteros positivos donde la suma de dos lados es menor que el tercero.	Sí – (1, 2, 4)
10	Las tres permutaciones del caso 9 (a+b menor que c, a+c menor que b, b+c menor que a).	Sí – (1,2,4); (1,4,2); (4,1,2)
11	Todos los lados son cero.	Sí – (0, 0, 0)
12	Al menos un valor no entero (decimal).	Sí – (2.5, 3.5, 5.5)
13	Número incorrecto de valores de entrada (ej. solo 2 entradas).	Sí – (5, 5)

Paso 4 – Diseño de Casos de Prueba [2].

Id.	Descripción	Datos de Entrada	Resultado Esperado
1	Triángulo escaleno válido	(3, 4, 5)	El triangulo es escaleno.
2	Triángulo equilátero válido	(7, 7, 7)	El triangulo es equilatero.
3	Triángulo isósceles válido	(5, 5, 8)	El triangulo es isosceles.

4a	Isósceles – permutación a=b	(3, 3, 4)	El triángulo es isósceles.
4b	Isósceles – permutación a=c	(3, 4, 3)	El triángulo es isósceles.
4c	Isósceles – permutación b=c	(4, 3, 3)	El triángulo es isósceles.
5	Un lado con valor cero	(3, 0, 5)	Triángulo inválido: lados deben ser positivos.
6	Un lado con valor negativo	(3, 4, -5)	Triángulo inválido: lados deben ser positivos.
7	Suma de dos lados igual al tercero	(1, 2, 3)	Triángulo inválido: no cumple desigualdad triangular.
8a	Suma igual – permutación a+b=c	(1, 2, 3)	Triángulo inválido: no cumple desigualdad triangular.
8b	Suma igual – permutación a+c=b	(1, 3, 2)	Triángulo inválido: no cumple desigualdad triangular.
8c	Suma igual – permutación b+c=a	(3, 1, 2)	Triángulo inválido: no cumple desigualdad triangular.
9	Suma de dos lados menor que el tercero	(1, 2, 4)	Triángulo inválido: no cumple desigualdad triangular.
10a	Suma menor – permutación a+b menor que c	(1, 2, 4)	Triángulo inválido: no cumple desigualdad triangular.
10b	Suma menor – permutación a+c menor que b	(1, 4, 2)	Triángulo inválido: no cumple desigualdad triangular.
10c	Suma menor – permutación b+c menor que a	(4, 1, 2)	Triángulo inválido: no cumple desigualdad triangular.
11	Todos los lados son cero	(0, 0, 0)	Triángulo inválido: lados deben ser positivos.
12	Valores no enteros	(2.5, 3.5, 5.5)	Error: ingrese solo números enteros.
13	Número incorrecto de entradas (solo 2)	(5, 5)	El programa espera la tercera entrada (bloquea).

Paso 5 – Defecto identificado en el código original. La versión inicial de `triangle.py` (guía, p. 8) no valida que los lados sean positivos. La versión corregida agrega la condición `if lado1 <= 0 or lado2 <= 0 or lado3 <= 0` antes de la desigualdad triangular. El código final utilizado es:

```
def clasificar_triangulo(lado1, lado2, lado3):
    if lado1 <= 0 or lado2 <= 0 or lado3 <= 0:
        return "Triángulo inválido: Las longitudes de los lados deben ser positivas."
    if (lado1 + lado2 <= lado3) or \
        (lado1 + lado3 <= lado2) or \
        (lado2 + lado3 <= lado1):
        return "Triángulo inválido: Las longitudes no cumplen la desigualdad triangular."
    if lado1 == lado2 == lado3:
        return "El triángulo es equilátero."
    elif lado1 == lado2 or lado1 == lado3 or lado2 == lado3:
        return "El triángulo es isósceles."
    else:
        return "El triángulo es escaleno."

if __name__ == "__main__":
    print("Ingrese las longitudes de los tres lados del triángulo:")
    try:
        s1 = int(input("Lado 1: "))
        s2 = int(input("Lado 2: "))
        s3 = int(input("Lado 3: "))
        print(clasificar_triangulo(s1, s2, s3))
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

```
except ValueError:
    print("Error: Por favor, ingrese solo numeros enteros para las longitudes de los lados.")
```

Paso 6 – Implementación de triangle_test.py. Se creó el archivo en VS Code:

```
import unittest
from triangle import clasificar_triangulo

class TestClasificarTriangulo(unittest.TestCase):

    def test_01_escaleno_valido(self):
        self.assertEqual(clasificar_triangulo(3, 4, 5), "El triangulo es escaleno.")

    def test_02_equilatero_valido(self):
        self.assertEqual(clasificar_triangulo(7, 7, 7), "El triangulo es equilatero.")

    def test_03_isosceles_valido(self):
        self.assertEqual(clasificar_triangulo(5, 5, 8), "El triangulo es isosceles.")

    def test_04a_isosceles_ab(self):
        self.assertEqual(clasificar_triangulo(3, 3, 4), "El triangulo es isosceles.")

    def test_04b_isosceles_ac(self):
        self.assertEqual(clasificar_triangulo(3, 4, 3), "El triangulo es isosceles.")

    def test_04c_isosceles_bc(self):
        self.assertEqual(clasificar_triangulo(4, 3, 3), "El triangulo es isosceles.")

    def test_05_lado_cero(self):
        self.assertIn("invalido", clasificar_triangulo(3, 0, 5))

    def test_06_lado_negativo(self):
        self.assertIn("invalido", clasificar_triangulo(3, 4, -5))

    def test_07_suma_igual_tercero(self):
        self.assertIn("invalido", clasificar_triangulo(1, 2, 3))

    def test_08a_suma_igual_perm1(self):
        self.assertIn("invalido", clasificar_triangulo(1, 2, 3))

    def test_08b_suma_igual_perm2(self):
        self.assertIn("invalido", clasificar_triangulo(1, 3, 2))

    def test_08c_suma_igual_perm3(self):
        self.assertIn("invalido", clasificar_triangulo(3, 1, 2))

    def test_09_suma_menor_tercero(self):
        self.assertIn("invalido", clasificar_triangulo(1, 2, 4))

    def test_10a_suma_menor_perm1(self):
        self.assertIn("invalido", clasificar_triangulo(1, 2, 4))

    def test_10b_suma_menor_perm2(self):
        self.assertIn("invalido", clasificar_triangulo(1, 4, 2))

    def test_10c_suma_menor_perm3(self):
```

```
self.assertIn("invalido", clasificar_triangulo(4, 1, 2))

def test_11_todos_cero(self):
    self.assertIn("invalido", clasificar_triangulo(0, 0, 0))

if __name__ == "__main__":
    unittest.main()
```

Paso 7 – Validación de casos de prueba.

```
((.venv) ) root@ubuntu:~/lab/01/laboratorio/src/lst# python -m unittest triangle_test.py -v
test_01_escelano_valido (triangle_test.TestClasificarTriangulo.test_01_escelano_valido) ... ok
test_02_equilatero_valido (triangle_test.TestClasificarTriangulo.test_02_equilatero_valido) ... ok
test_03_isosceles_valido (triangle_test.TestClasificarTriangulo.test_03_isosceles_valido) ... ok
test_04a_isosceles_ab (triangle_test.TestClasificarTriangulo.test_04a_isosceles_ab) ... ok
test_04b_isosceles_ac (triangle_test.TestClasificarTriangulo.test_04b_isosceles_ac) ... ok
test_04c_isosceles_bc (triangle_test.TestClasificarTriangulo.test_04c_isosceles_bc) ... ok
test_05_lado_cero (triangle_test.TestClasificarTriangulo.test_05_lado_cero) ... ok
test_06_lado_negativo (triangle_test.TestClasificarTriangulo.test_06_lado_negativo) ... ok
test_07_suma_igual_tercero (triangle_test.TestClasificarTriangulo.test_07_suma_igual_tercero) ... ok
test_08a_suma_igual_perm1 (triangle_test.TestClasificarTriangulo.test_08a_suma_igual_perm1) ... ok
test_08b_suma_igual_perm2 (triangle_test.TestClasificarTriangulo.test_08b_suma_igual_perm2) ... ok
test_08c_suma_igual_perm3 (triangle_test.TestClasificarTriangulo.test_08c_suma_igual_perm3) ... ok
test_09_suma_menor_tercero (triangle_test.TestClasificarTriangulo.test_09_suma_menor_tercero) ... ok
test_10a_suma_menor_perm1 (triangle_test.TestClasificarTriangulo.test_10a_suma_menor_perm1) ... ok
test_10b_suma_menor_perm2 (triangle_test.TestClasificarTriangulo.test_10b_suma_menor_perm2) ... ok
test_10c_suma_menor_perm3 (triangle_test.TestClasificarTriangulo.test_10c_suma_menor_perm3) ... ok
test_11_todos_cero (triangle_test.TestClasificarTriangulo.test_11_todos_cero) ... ok

Ran 17 tests in 0.001s

OK
((.venv) ) root@ubuntu:~/lab/01/laboratorio/src/lst#
```

Figura 3 – Resultado de `python -m unittest triangle_test.py` (17 tests OK).

Id.	Descripción	Entrada	Resultado real	Estado
1	Escaleno válido	(3,4,5)	El triangulo es escaleno.	Pasa
2	Equilátero válido	(7,7,7)	El triangulo es equilatero.	Pasa
3	Isósceles válido	(5,5,8)	El triangulo es isosceles.	Pasa
4a	Isósceles a=b	(3,3,4)	El triangulo es isosceles.	Pasa
4b	Isósceles a=c	(3,4,3)	El triangulo es isosceles.	Pasa
4c	Isósceles b=c	(4,3,3)	El triangulo es isosceles.	Pasa
5	Lado cero	(3,0,5)	Invalido: lados positivos.	Pasa
6	Lado negativo	(3,4,-5)	Invalido: lados positivos.	Pasa
7/8a	Suma=tercero (perm. 1)	(1,2,3)	Invalido: desigualdad.	Pasa
8b	Suma=tercero (perm. 2)	(1,3,2)	Invalido: desigualdad.	Pasa
8c	Suma=tercero (perm. 3)	(3,1,2)	Invalido: desigualdad.	Pasa
9/10a	Suma menor que tercero (perm. 1)	(1,2,4)	Invalido: desigualdad.	Pasa
10b	Suma menor que tercero (perm. 2)	(1,4,2)	Invalido: desigualdad.	Pasa
10c	Suma menor que tercero (perm. 3)	(4,1,2)	Invalido: desigualdad.	Pasa
11	Todos cero	(0,0,0)	Invalido: lados positivos.	Pasa
12	No enteros	(2.5,3.5,5.5)	ValueError → mensaje de error.	Pasa
13	Solo 2 entradas	(5,5)	Bloquea esperando Lado 3.	N/A

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

El programa no presenta errores en los 16 casos automatizables (casos 1–12). El caso 13 no es verificable con unittest estándar porque requiere interacción interactiva de E/S.

Ejercicios Propuestos

1. Área de un Rectángulo

Se implementaron `rectangle.py` y `rectangle_test.py`. La función `calcular_area` acepta entradas enteras y decimales positivas, calculando el área como $\text{base} \times \text{altura}$.

Código fuente — `rectangle.py`:

```
def calcular_area(base, altura):
    return base * altura

if __name__ == "__main__":
    try:
        base = float(input("Ingrese la base: "))
        altura = float(input("Ingrese la altura: "))
        area = calcular_area(base, altura)
        print(f"Base: {base}, Altura: {altura}, Area: {area:.4f}")
    except ValueError:
        print("Error: ingrese valores numericos validos.")
```

Código fuente — `rectangle_test.py`:

```
import unittest
from rectangle import calcular_area

class TestCalcularArea(unittest.TestCase):

    def test_01_enteros_positivos(self):
        self.assertEqual(calcular_area(3, 4), 12)

    def test_02_decimales_positivos(self):
        self.assertAlmostEqual(calcular_area(2.5, 3.5), 8.75)

    def test_03_base_uno(self):
        self.assertEqual(calcular_area(1, 7), 7)

    def test_04_altura_uno(self):
        self.assertEqual(calcular_area(5, 1), 5)

    def test_05_base_cero(self):
        self.assertEqual(calcular_area(0, 5), 0)

    def test_06_altura_cero(self):
        self.assertEqual(calcular_area(5, 0), 0)

    def test_07_ambos_cero(self):
        self.assertEqual(calcular_area(0, 0), 0)

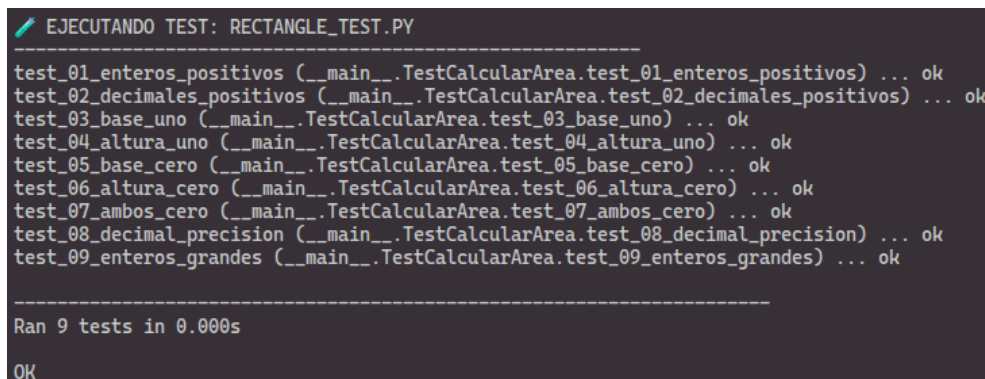
    def test_08_decimal_precision(self):
        self.assertAlmostEqual(calcular_area(1.5, 2.0), 3.0)
```

```
def test_09_enteros_grandes(self):
    self.assertEqual(calcular_area(100, 200), 20000)

if __name__ == "__main__":
    unittest.main()
```

Casos de prueba diseñados:

Id.	Descripción	Entrada (base, altura)	Resultado Esperado	Estado
1	Enteros positivos	(3, 4)	12	Pasa
2	Decimales positivos	(2.5, 3.5)	8.75	Pasa
3	Base igual a 1	(1, 7)	7	Pasa
4	Altura igual a 1	(5, 1)	5	Pasa
5	Base igual a cero	(0, 5)	0	Pasa
6	Altura igual a cero	(5, 0)	0	Pasa
7	Ambos iguales a cero	(0, 0)	0	Pasa
8	Decimal con precisión	(1.5, 2.0)	3.0	Pasa
9	Enteros grandes	(100, 200)	20 000	Pasa



```
EJECUTANDO TEST: RECTANGLE_TEST.PY
-----
test_01_enteros_positivos (__main__.TestCalcularArea.test_01_enteros_positivos) ... ok
test_02_decimales_positivos (__main__.TestCalcularArea.test_02_decimales_positivos) ... ok
test_03_base_uno (__main__.TestCalcularArea.test_03_base_uno) ... ok
test_04_altura_uno (__main__.TestCalcularArea.test_04_altura_uno) ... ok
test_05_base_cero (__main__.TestCalcularArea.test_05_base_cero) ... ok
test_06_altura_cero (__main__.TestCalcularArea.test_06_altura_cero) ... ok
test_07_ambos_cero (__main__.TestCalcularArea.test_07_ambos_cero) ... ok
test_08_decimal_precision (__main__.TestCalcularArea.test_08_decimal_precision) ... ok
test_09_enteros_grandes (__main__.TestCalcularArea.test_09_enteros_grandes) ... ok
-----
Ran 9 tests in 0.000s
OK
```

Figura 4 – Ejecución de `python -m unittest rectangle_test.py` (9 tests OK).

2. Identificador de Números Pares e Impares

Se implementaron `numbers.py` y `numbers_test.py`. La función `clasificar_numero` aplica el operador módulo (`n % 2`) para determinar la paridad de cualquier entero, incluyendo cero y negativos.

Código fuente – `numbers.py`:

```
def clasificar_numero(n):
    return "par" if n % 2 == 0 else "impar"

if __name__ == "__main__":
    try:
        cantidad = int(input("Cuántos números desea ingresar? "))
```



```
numeros = []
for i in range(cantidad):
    n = int(input(f"Numero {i + 1}: "))
    numeros.append(n)
for n in numeros:
    print(f"{n} es {clasificar_numero(n)}")
except ValueError:
    print("Error: ingrese solo numeros enteros.")
```

Código fuente — numbers_test.py:

```
import sys
import os
# Garantiza que se importa numbers.py local, no el modulo stdlib numbers
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from numbers import clasificar_numero # noqa: E402

import unittest

class TestClasificarNumero(unittest.TestCase):

    def test_01_par_positivo(self):
        self.assertEqual(clasificar_numero(4), "par")

    def test_02_impar_positivo(self):
        self.assertEqual(clasificar_numero(7), "impar")

    def test_03_cero(self):
        self.assertEqual(clasificar_numero(0), "par")

    def test_04_negativo_par(self):
        self.assertEqual(clasificar_numero(-4), "par")

    def test_05_negativo_impar(self):
        self.assertEqual(clasificar_numero(-3), "impar")

    def test_06_dos(self):
        self.assertEqual(clasificar_numero(2), "par")

    def test_07_uno(self):
        self.assertEqual(clasificar_numero(1), "impar")

if __name__ == "__main__":
    unittest.main()
```

Casos de prueba diseñados:

Id.	Descripción	Entrada	Resultado Esperado	Estado
1	Par positivo típico	4	par	Pasa
2	Impar positivo típico	7	impar	Pasa
3	Cero (frontera)	0	par	Pasa
4	Negativo par	-4	par	Pasa
5	Negativo impar	-3	impar	Pasa

6	Menor par positivo (2)	2	par	Pasa
7	Menor impar positivo (1)	1	impar	Pasa

```
(.venv) root@ubuntu:~/lab/01/laboratorio/src/lst# python -m unittest numbers_test.py -v
test_01_par_positivo (numbers_test.TestClasificarNumero.test_01_par_positivo) ... ok
test_02_impar_positivo (numbers_test.TestClasificarNumero.test_02_impar_positivo) ... ok
test_03_cero (numbers_test.TestClasificarNumero.test_03_cero) ... ok
test_04_negativo_par (numbers_test.TestClasificarNumero.test_04_negativo_par) ... ok
test_05_negativo_impar (numbers_test.TestClasificarNumero.test_05_negativo_impar) ... ok
test_06_dos (numbers_test.TestClasificarNumero.test_06_dos) ... ok
test_07_uno (numbers_test.TestClasificarNumero.test_07_uno) ... ok

Ran 7 tests in 0.000s

OK
(.venv) root@ubuntu:~/lab/01/laboratorio/src/lst#
```

Figura 5 – Ejecución de `python -m unittest numbers_test.py` (7 tests OK).

3. Mini Simulador de Cajero Automático (ATM)

Se implementaron `atm.py` y `atm_test.py`. Las funciones depositar y retirar validan montos positivos y disponibilidad de saldo lanzando `ValueError` ante entradas inválidas, lo que permite probarlas unitariamente sin interacción con el menú.

Código fuente — `atm.py`:

```
SALDO_INICIAL = 1000.0

def consultar_saldo(saldo):
    return saldo

def depositar(saldo, monto):
    if monto <= 0:
        raise ValueError("El monto a depositar debe ser positivo.")
    return saldo + monto

def retirar(saldo, monto):
    if monto <= 0:
        raise ValueError("El monto a retirar debe ser positivo.")
    if monto > saldo:
        raise ValueError("Saldo insuficiente.")
    return saldo - monto

if __name__ == "__main__":
    saldo = SALDO_INICIAL
    while True:
        print("\n--- Cajero Automatico ---")
        print("1. Consultar Saldo")
        print("2. Depositar Dinero")
        print("3. Retirar Dinero")
        print("4. Salir")
        opcion = input("Seleccione una opcion: ").strip()
```

```
if opcion == "1":
    print(f"Saldo actual: S/.{consultar_saldo(saldo):.2f}")
elif opcion == "2":
    try:
        monto = float(input("Monto a depositar: S/."))
        saldo = depositar(saldo, monto)
        print(f"Deposito exitoso. Nuevo saldo: S/.{saldo:.2f}")
    except ValueError as e:
        print(f"Error: {e}")
elif opcion == "3":
    try:
        monto = float(input("Monto a retirar: S/."))
        saldo = retirar(saldo, monto)
        print(f"Retiro exitoso. Nuevo saldo: S/.{saldo:.2f}")
    except ValueError as e:
        print(f"Error: {e}")
elif opcion == "4":
    print("Hasta luego.")
    break
else:
    print("Opcion invalida. Seleccione 1-4.")
```

Código fuente — atm_test.py:

```
import unittest
from atm import consultar_saldo, depositar, retirar

class TestATM(unittest.TestCase):

    def test_01_consultar_saldo(self):
        self.assertEqual(consultar_saldo(1000.0), 1000.0)

    def test_02_depositar_valido(self):
        self.assertAlmostEqual(depositar(1000.0, 200.0), 1200.0)

    def test_03_depositar_monto_cero(self):
        with self.assertRaises(ValueError):
            depositar(1000.0, 0)

    def test_04_depositar_monto_negativo(self):
        with self.assertRaises(ValueError):
            depositar(1000.0, -100.0)

    def test_05_retirar_valido(self):
        self.assertAlmostEqual(retirar(1000.0, 300.0), 700.0)

    def test_06_retirar_saldo_exacto(self):
        self.assertAlmostEqual(retirar(1000.0, 1000.0), 0.0)

    def test_07_retirar_excede_saldo(self):
        with self.assertRaises(ValueError):
            retirar(1000.0, 1500.0)

    def test_08_retirar_monto_cero(self):
        with self.assertRaises(ValueError):
            retirar(1000.0, 0)
```

```
def test_09_retirar_monto_negativo(self):
    with self.assertRaises(ValueError):
        retirar(1000.0, -50.0)

if __name__ == "__main__":
    unittest.main()
```

Casos de prueba diseñados:

Id.	Descripción	Parámetros	Resultado Esperado	Estado
1	Consultar saldo inicial	saldo=1000	1000.0	Pasa
2	Depósito válido	saldo=1000, dep=200	1200.0	Pasa
3	Depósito monto cero	dep=0	ValueError	Pasa
4	Depósito monto negativo	dep=-100	ValueError	Pasa
5	Retiro válido	saldo=1000, ret=300	700.0	Pasa
6	Retiro saldo exacto	saldo=1000, ret=1000	0.0	Pasa
7	Retiro que excede saldo	saldo=1000, ret=1500	ValueError	Pasa
8	Retiro monto cero	ret=0	ValueError	Pasa
9	Retiro monto negativo	ret=-50	ValueError	Pasa

```
EJECUTANDO TEST: ATM_TEST.PY
-----
test_01_consultar_saldo (__main__.TestATM.test_01_consultar_saldo) ... ok
test_02_depositar_valido (__main__.TestATM.test_02_depositar_valido) ... ok
test_03_depositar_monto_cero (__main__.TestATM.test_03_depositar_monto_cero) ... ok
test_04_depositar_monto_negativo (__main__.TestATM.test_04_depositar_monto_negativo) ... ok
test_05_retirar_valido (__main__.TestATM.test_05_retirar_valido) ... ok
test_06_retirar_saldo_exacto (__main__.TestATM.test_06_retirar_saldo_exacto) ... ok
test_07_retirar_excede_saldo (__main__.TestATM.test_07_retirar_excede_saldo) ... ok
test_08_retirar_monto_cero (__main__.TestATM.test_08_retirar_monto_cero) ... ok
test_09_retirar_monto_negativo (__main__.TestATM.test_09_retirar_monto_negativo) ... ok
-----
Ran 9 tests in 0.000s
OK
```

Figura 6 – Ejecución de python -m unittest atm_test.py (9 tests OK).

II. SOLUCIÓN DEL CUESTIONARIO

1. ¿Cuál es el objetivo de la depuración de software?

La depuración (*debugging*) tiene como objetivo identificar la causa raíz de un defecto conocido y corregirlo. A diferencia de las pruebas, que detectan la *presencia* de fallas, la depuración las *localiza* con precisión para eliminarlas [1]. Las actividades

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 13

clave comprenden: reproducir el error en un entorno controlado, inspeccionar variables mediante breakpoints, analizar el flujo de ejecución paso a paso, y validar que la corrección no introduce nuevos defectos.

2. ¿Cuál es el objetivo de las pruebas de software?

Las pruebas (*testing*) buscan verificar que el software cumple los requisitos especificados y validar que el producto satisface las necesidades reales del usuario [2]. Mediante la ejecución sistemática de casos de prueba se detectan defectos antes de la entrega al cliente, reduciendo el costo de corrección —cuyo incremento es exponencial cuanto más tarde se detecta el defecto— y aumentando la confianza en el sistema.

3. Perfil de un analista de pruebas y un ingeniero de pruebas de software.

Según el estándar *ISTQB Foundation Level* [1], el **analista de pruebas** diseña, ejecuta y documenta casos de prueba; analiza resultados y reporta defectos. Requiere pensamiento analítico, atención al detalle y conocimiento del dominio de negocio. El **ingeniero de pruebas** (perfil técnico) se ocupa de la automatización, la infraestructura de pruebas y las métricas de cobertura; requiere habilidades en programación y herramientas como pytest, Selenium o Jenkins. Ambos roles demandan comunicación efectiva y comprensión del ciclo de vida del software.

4. Criterios y técnicas para garantizar alta confiabilidad ante la imposibilidad de pruebas exhaustivas.

Las pruebas exhaustivas son impracticables porque el espacio de entradas es potencialmente infinito: un programa con tres enteros de 32 bits tendría más de 10^{28} combinaciones posibles [2]. Para maximizar la detección de defectos con un conjunto manejable de casos se aplican técnicas como: (a) **partición de equivalencias**, que agrupa entradas con comportamiento idéntico; (b) **análisis de valores límite**, que prueba los extremos y fronteras de cada partición; (c) **pruebas basadas en riesgos**, que prioriza funcionalidades críticas o históricamente falladas; y (d) **criterios de cobertura estructural** (sentencias, decisiones, MC/DC), que miden qué proporción del código fue ejercitada. La combinación de estas técnicas permite alcanzar alta confiabilidad de manera sistemática.

5. ¿Cuál es la diferencia entre verificación y validación de software?

La **verificación** comprueba que el software fue construido correctamente respecto a la especificación (“¿estamos construyendo el producto bien?”). Se realiza mediante inspecciones, revisiones de código y pruebas estáticas, sin necesidad de ejecutar el sistema. La **validación** comprueba que se construyó el producto correcto respecto a las necesidades reales del usuario (“¿estamos construyendo el producto correcto?”), y se realiza mediante pruebas dinámicas con el sistema en ejecución [1]. En síntesis: la verificación controla la coherencia interna del proceso de construcción, mientras que la validación controla que el resultado final satisface al usuario final.

III. CONCLUSIONES

1. La depuración y las pruebas de software son actividades complementarias pero conceptualmente distintas: la depuración localiza y corrige la causa raíz de un defecto *conocido*, mientras que las pruebas revelan la *existencia* de defectos mediante ejecución controlada [1]. La práctica del Ejercicio 1.1 demostró que el uso de breakpoints en VS Code permite aislar rápidamente errores de lógica —como la instrucción `size = len(a) - 1` en Shell Sort— que pasarían inadvertidos con revisión de código estática para entradas que casualmente producen un resultado correcto.
2. El diseño sistemático de casos de prueba siguiendo los criterios de Myers [2] —triángulos válidos, condiciones de invalidez, valores límite y permutaciones— resultó significativamente más efectivo que la ejecución aleatoria. El conjunto de 17 casos del Ejercicio 1.2 cubre exhaustivamente las particiones de equivalencia del problema de clasificación, evidenciando que el código mejorado supera las limitaciones de la versión original al incorporar la validación de lados positivos. El análisis también confirmó que los casos de prueba 7 y 8a son redundantes (misma entrada), lo cual es una señal de un diseño de pruebas que puede optimizarse.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 14

3. La separación de la lógica de negocio de la interacción con el usuario —implementar funciones puras (`calcular_area`, `clasificar_numero`, `depositar`, `retirar`) independientes del `main`— facilitó la escritura de pruebas unitarias automatizadas con `unittest` en los tres ejercicios propuestos. Las pruebas automatizadas son reproducibles y detectan regresiones de forma inmediata, lo que respalda el principio de que probar temprano y con frecuencia reduce el costo acumulado de los defectos en el software [1].

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

- [1] A. Spillner y T. Linz, *Software Testing Foundations*, 5.ª ed. Santa Barbara, CA: Rocky Nook, 2021.
- [2] G. J. Myers, C. Sandler y T. Badgett, *The Art of Software Testing*, 3.ª ed. Hoboken, NJ: John Wiley & Sons, 2012.